Telematics Platform Writer's Guide
for Update Agent

Document Information	
Filename	TP_F100W-00015_Writers_Guide_Update_Agent
Status	Release
Revision Date	Sep 30, 2021
Version	09.00
Document ID	TP_F100W-00015

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Table of contents

1 Introduction ... 3
1.1 Notational Conventions ... 3
2 Setup ... 4
2.1 **Tools/Equipment Needed ... 4**
3 Components/Plugins ... 4
3.1 Update Agent ... 4
3.1.1 Overview ... 4
3.1.2 Interfaces ... 9
3.1.2.1 GetRegistrationInformation ... 13
3.1.2.2 GetCapabilities ... 13
3.1.2.3 GetTimeout ... 14
3.1.2.4 CheckPrerequisites ... 15
3.1.2.5 StartUaUpdate ... 16
3.1.2.6 GetAttributeRequest ... 17
3.1.2.7 PrepareForReset ... 17
3.1.2.8 UaVerifyImages ... 18
3.1.2.9 UaUpdateOutcome ... 18
3.1.2.10 UaCompleteUpdate ... 19
3.1.2.11 UaCompleteUpdateWithVerification ... 20
3.1.2.12 UaPrepareForUpdate ... 20
3.1.2.13 StartUaUpdatePostInstall ... 21
3.1.2.14 UaVerifySoftwareActivation ... 22
3.1.2.15 UaGetImageData ... 23
3.1.2.16 PrerequisitesStatus ... 23
3.1.2.17 UaUpdateStatus ... 24
3.1.2.18 UaPrepareForResetStatus ... 24
3.1.2.19 UaUpdateOutcomeStatus ... 24
3.1.2.20 UaCompleteUpdateStatus ... 26
3.1.2.21 UaVerifyImagesResult ... 26
4 FAQs ... 27
5 References ... 27
5.1 Abbreviations/Acronyms/Definitions ... 27
5.2 Platform Documentation ... 27
5.3 Website links ... 27
6 Revision History ... 28

© Continental Automotive GmbH 2019
Confidential and Proprietary

1 Introduction

This document provides instructions for implementing Update Agent to provide BSP access to the Continental Telematics Framework/Middleware. This document does not detail the Software Loading Manager's usage of an Update Agent, which is covered in a separate Software Loading Manager user guide.

The Update Agent provides facilities to interact with the Software Loading Manager, which controls the update process and handles requests from the Project Application. An Update Agent registers with the Software Loading Manager to indicate that it is capable of handling a specific element in the system. The Update Agent performs the verification and update operations for the specific element.

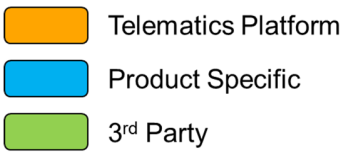
This writer's guide is intended for a developer wanting to write an Update Agent. If the intent is to use the Update Agent and existing functionality then the Software Loading Manager user guide can be referenced. An Update Agent allows a developer to extend the current Software Loading Manager functionality. If the system requires to update an unsupported element, then an Update Agent should be written for it.

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



© Continental Automotive GmbH 2019
Confidential and Proprietary

2 Setup

1. *Update Agents must register to DBus with service name prefix* ***com.contiautomotive.softwareloading.updateagent.[modemap|extap].[name]***
   a. *Name should be a unique name (not special characters).*
2. *Update Agents must be part of "**swl-ua**" group in order own the DBus service name.*
3. *Update Agents must be registered at least to one content type.*
4. *Two Update Agents should not be registered to the same content type.*
5. Include Software Loading UA Signals library header files in Update Agent source code
   `#include "swl_ua_signal_utilities.h"`
6. Use pkg-config in the client application Makefile to link to the Software Loading Manager client library
   `LDFLAGS += `pkg-config -libs swl-ua-signals-utils``
   `CFLAGS += `pkg-config -cflags swl-ua-signals-utils ``

2.1 Tools/Equipment Needed

Not Applicable.

3 Components/Plugins

3.1 Update Agent

3.1.1 Overview

The update process takes care of updating elements in a target. A target is an OEM HW. The elements that can be updated are any of the SW running on such HW and are called "updatable elements". The update process requires that the Software Loading Manager handles a special file called "update package" which is installed on the target by means of a Download Client. The "update package" is a container that stores following elements:

- Image files: A file that is intended to be applied on an "updatable element"
- Signature files: A file that is intended to be used to verify the integrity of an image
- update_package_descriptor.xml: A special file containing metadata about images, updatable elements and other information relevant to the update process.

The Software Loading Manager extracts those elements from the "update package", then only the image files and the **update_package_descriptor.xml** are passed to the Update Agent. Signature files are outside of the scope of this document and won't be addressed. Update Agents can support different update workflows depending on the upgrade mode as defined by `tpSWL_UpgradeMode_t`.

The Update Agent uses the image files as the data to be applied on the "updatable elements" and the **update_package_descriptor.xml** as the guide to understand how to apply the images files. In this file, it is provided information about the version to be updated, the image file name, update type and the content type (among others). The **content type** is a unique name that is assigned to an element (or set of elements) to identify them with the Software Loading Manager which then will proceed to match the content type with an Update Agent that is registered to it. When an Update Agent is developed, the developer should consider that the Update Agent should be capable of handling one, or multiple, **content type**(s).

An important note is that a **content type** should not be handled by multiple Update Agents at the same time since it will cause both of them to conflict.

© Continental Automotive GmbH 2019
Confidential and Proprietary

The Update Agent allows that multiple **content types** can be updated in the same product. An Update Agent shall report which content type is supported by it and therefore apply the update when requested by the Software Loading Manager. This is accomplished using DBus signals and events. DBus is a message bus system, meaning that applications can talk to each other by using it. DBus itself has a concept called "DBus interface file" which is an XML file which describes one or more DBus interfaces.

Systemd is responsible for starting processes during boot and stopping processes during shutdown. It is recommended to use systemd to control the lifecycle of Update Agent processes.

Update Agents must support rollback capabilities. This can be achieved by using Double Banking or a backup (Single Banking).

Double Banking SW Update is a mechanism where a second copy of an element is retained indefinitely after the SW update, normally to allow a return to the (older) version either for a failsafe or rollback mechanism. If an Update Agent supports Double Banking, the component must be updated in the inactive Bank.

If an Update Agent only supports Single Banking, a backup of the component must be created to be used during rollback process.

The Update Agent should be prepared to receive an identical image to the one currently executing in the system (samegrade).

The Update Agent might modify its behavior by monitoring the expectations on the system by checking the contents of the persistence key swl_allow_same_version_write (see TP_F101U-00028), however is up to the update agent to change its behavior based on its defined value.

- If disabled there is no expectation on how to handle a samegrade.
- If enabled the expectation would be for the update agent to exercise the writing process of the image on a samegrade to behave as it it is an upgrade.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 1 Update Agent system interaction shows how the components coexist in the system using DBus and systemd.

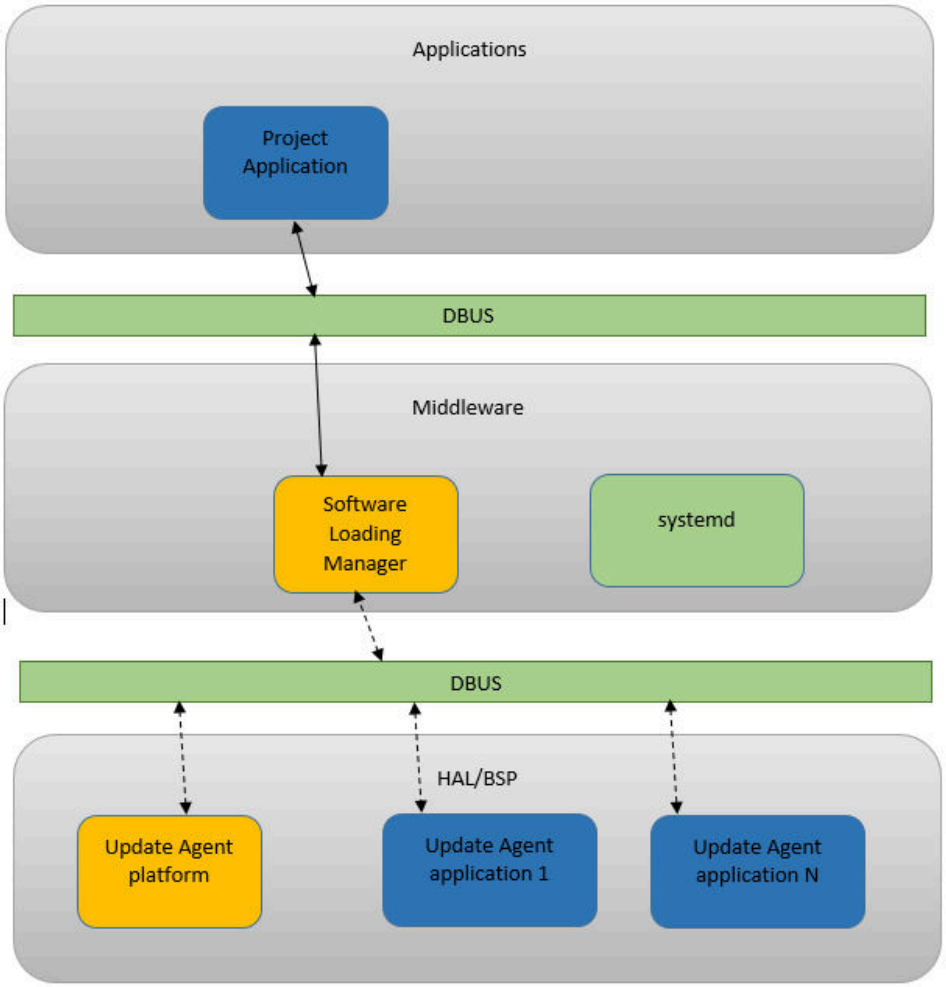


Figure 1 Update Agent system interaction

Systemd is a key element of the Update Agent since it indicates at what time it should start its initializations, its dependencies and under which operation mode it runs. In the context of systemd, an operation mode is a special folder in the file system where a file called a unit file it is used to understand under which conditions an Update Agent will run. In a typical Linux filesystem, these folders are located under /lib/systemd/system and the available ones are:

- multi-user.target.wants: Normal operation mode, Update Agent is expected to run in this mode to perform prerequisite checks to ensure **content type** elements integrity and update operations.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Finally, each element contained in the **update_package_descriptor.xml** is described in below example.

Tag Name	Description	Structure
packagecontent	Root node containing the metadata information used by Update Agents to identify updatable elements in the update package.	Root node.
numberimages	Specifies the number of image files to be used.	Attribute of **packagecontent** node.
oem	Node which contains OEM metadata.	Sub-Node of **packagecontent** node.
version	Attribute for indicating the OEM version to be updated.	Attribute of oem node.
image	Node which contains image file metadata.	Sub-Node of **packagecontent** node.
name	Image file unique identifier	Attribute of **image** node.
filename	Image file name	Attribute of **image** node.
type	Image file type, which can be used by Update Agent to identify the update type.	Attribute of **image** node.
size	Image file size in the filesystem.	Sub-Node of **image** node.
newromsize	Expected new size applied in the updatable element after the image file is applied to it.	Sub-Node of **image** node.
contenttype	Unique name that is assigned to an element (or set of elements) to be updated.	Sub-Node of **image** node.
type	Whether the update is differential diff or full. Possible values are: - diff - full	Sub-Node of **image** node.
dependentfiles	Expected files to be handle before the update is executed.	Sub-Node of **image** node.
version	Updatable element target version to be applied after the update process.	Sub-Node of **image** node.
checksum	Unique value to verify image integrity.	Sub-Node of **image** node.
prerequisiteversion	Expected base version to contain before the update takes place.	Sub-Node of **image** node.
updatemode	Only uses SSU mode. Possible values are: - normal	Sub-Node of **image** node.
signatures	Node which contains signature file metadata for all the signature files.	Sub-Node of **packagecontent** node.
file	Node which contains signature file metadata for one single signature file.	Sub-Node of **signatures** node.
name	Image file name related to the signature file.	Attribute of **file** node.
signature	Signature file name.	Attribute of **file** node.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Here is an example of an **update_package_descriptor.xml**

```
<packagecontent numberimages="1">
	<oem version="OEM_NAME" />
	<image name="boot" filename="boot.img" type="mtd">
		<size>5103616</size>
		<newromsize>5103616</newromsize>
		<contenttype>boot</contenttype>
		<type>full</type>
		<dependentfiles>
		<!-- Set the dependent files here if any-->
		</dependentfiles>
		<version>2.0.1</version>
		<checksum>0xc3fe82ec</checksum>
		<prerequisiteversion>2.0.0</prerequisiteversion>
		<!-- Set this to tag to normal if needed to run in SSU -->
		<updatemode>normal</updatemode>
		<!-- Add project specific tags here -->
	</image>
	<signatures>
		<file name="boot.img" signature="boot.img.sig"/>
	</signatures>
</packagecontent>
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.2 Interfaces

All DBus methods and signals described in this section are derived from the DBus introspect file **software_loading_update_agent.xml**. DBus methods are expected to behave synchronously. When the DBus method returns all necessary operations are complete.

Note: Update Agent should reference "$(SDK)/otp/conti/share/interface" to access the DBus xml interface.

Type Description	Doxygen Path
DBus introspect methods	Modules |-> Software Loading |-> **Update Agent (UA)**
DBus introspect signals	Modules |-> Software Loading |-> **Update Agent (UA)**
Enumeration of error return codes	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_Error_t**
Enumeration of update status codes	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_UpdateStatus_t**
Enumeration of upgrade modes	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_UpgradeMode_t**
Enumeration of checksum status codes	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_ChecksumStatus_t**
Enumeration of component attributes	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_ComponentAttribute_t**
Enumeration of Update Agent capabilities	Modules |-> Software Loading |-> Software Loading Manager (SWL) |-> **tpSWL_UaCapabilities_t**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 2 Update Agent failure scenario for SSU upgrade mode shows the interaction between an Update Agent and the Software Loading Manager when the update process fails at prerequisite check stage. Note that update process is interrupted when a failure is reported from one Update Agent.

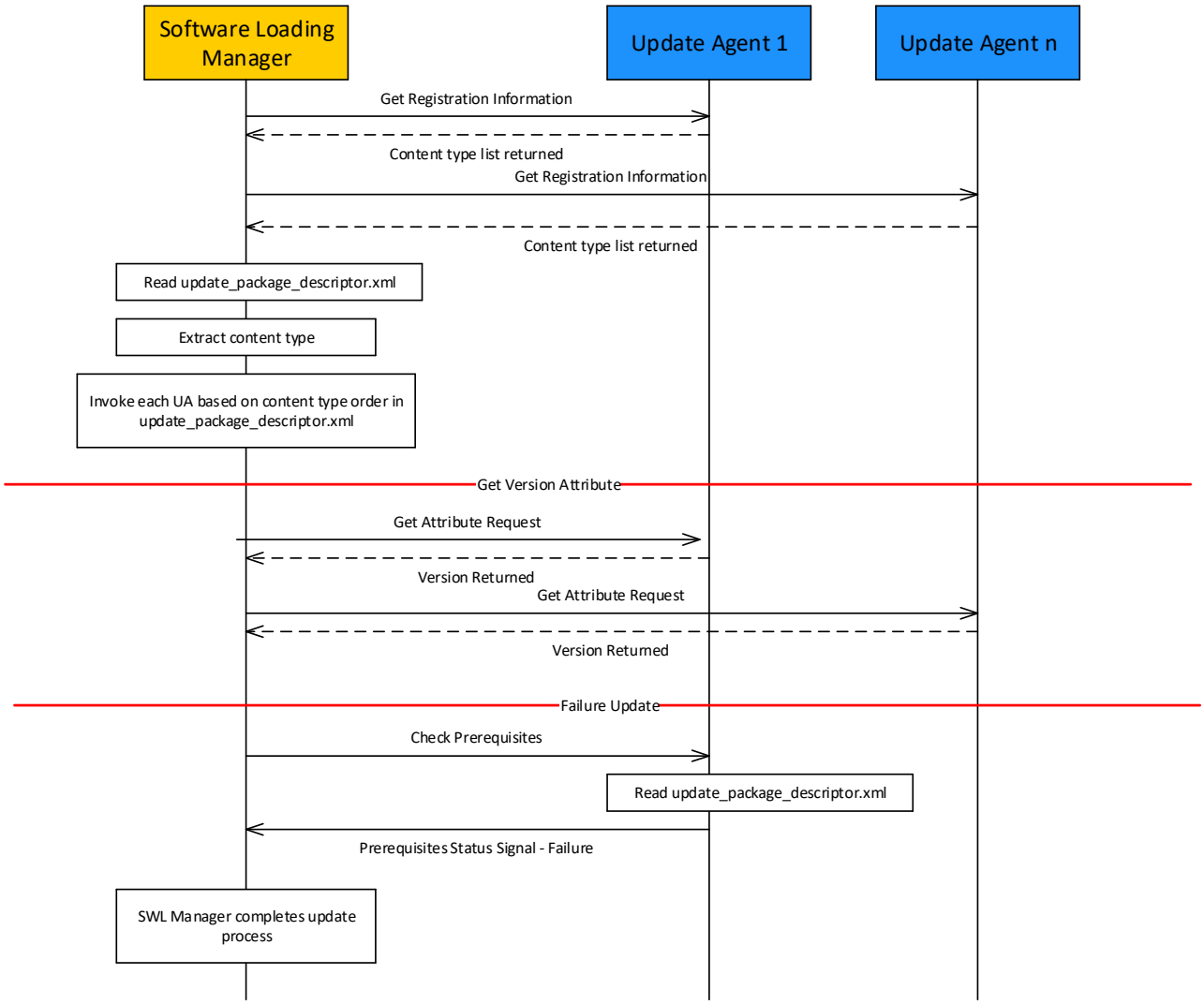
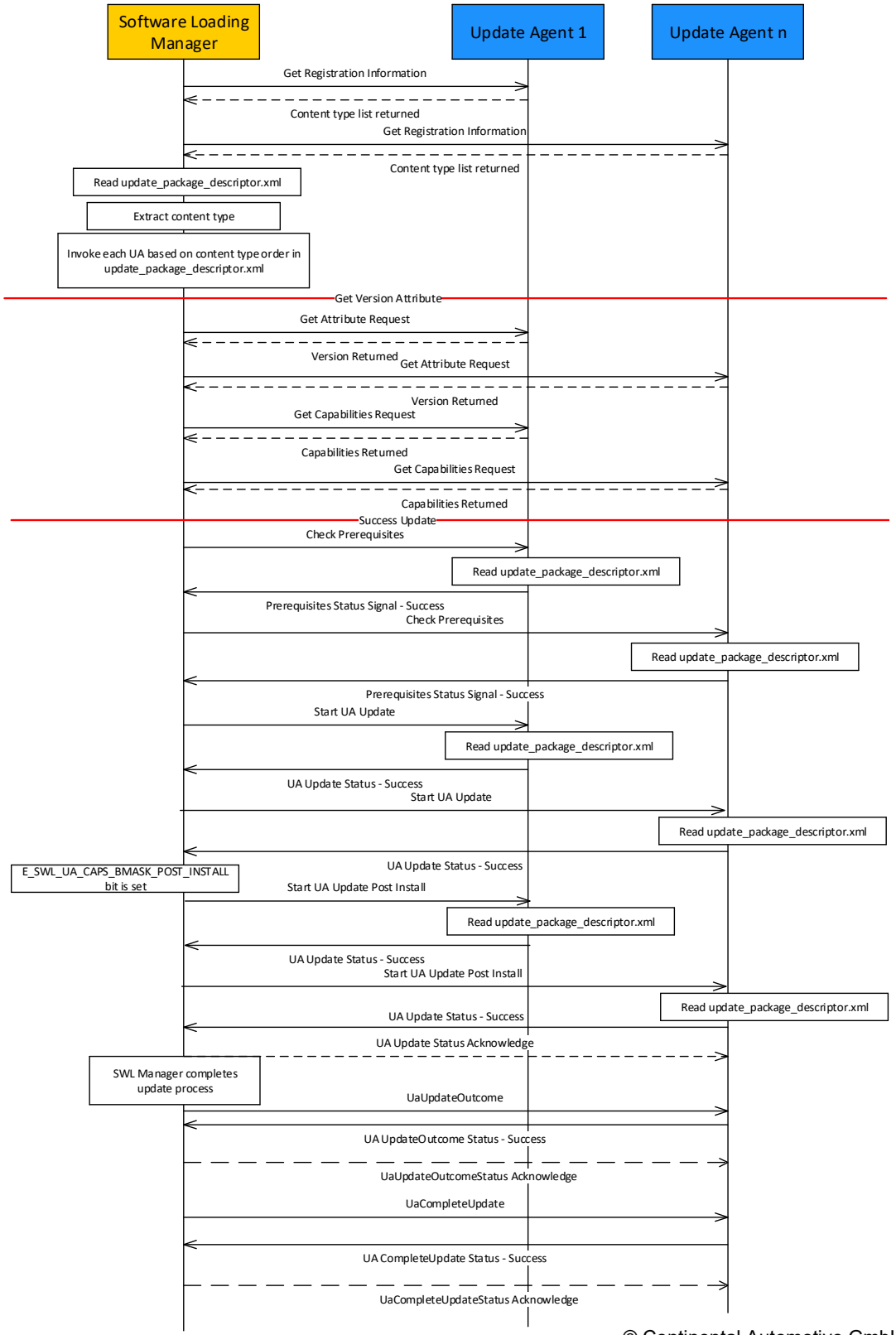


Figure 2 Update Agent failure scenario for SSU upgrade mode

© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 3 Update Agent success scenario for SSU upgrade mode shows the interaction between an Update Agent "X" and the Software Loading Manager when the update process is carried successfully for SSU upgrade mode, that sequence can be repeated with all the other existent Update Agents.

© Continental Automotive GmbH 2019
Confidential and Proprietary



© Continental Automotive GmbH 2019
Confidential and Proprietary

Figure 3 Update Agent success scenario for SSU upgrade mode

3.1.2.1 GetRegistrationInformation

Type: DBus Method

*Implementation **Required.***

Parameter	Type	Direction	Description
`content_type_support`	ARRAY OF COMPLEX "a(sb)"	Output	A list of strings containing the **content types** that the Update Agent can update.
`ua_name`	STRING "s"	Output	The unique identifier of the Update Agent.

This DBus method reports the **content types** that an Update Agent supports. It returns a list of **content types** that can be updated by the Update Agent and its unique name. The list of **content types** indicates that the Update Agent is capable of updating them and, using this list, the Software Loading Manager interacts with it. The Project Application indicates which **content types** are required using the **update_package_descriptor.xml** file which contains the "**contenttype**" tag. As shown below:

```
<contenttype>element</contenttype>
```

Based on this information, Software Loading Manager can correctly invoke an Update Agent capable of handling the **content types** specified by the **update_package_descriptor.xml**

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.2 GetCapabilities

Type: DBus Method

*Implementation **Required.***

Parameter	Type	Direction	Description
`capabilities`	UINT64 "t"	output	Supported capabilities

This DBus method reports the capabilities that an Update Agent has in regards to the update process. It returns a bitmask of the supported capabilities by the Update Agent. The possible values to use for the bitmask are defined by in `tpSWL_UaCapabilities_t` type.

© Continental Automotive GmbH 2019
Confidential and Proprietary

If an Update Agent has `E_SWL_UA_CAPS_BMASK_UPDATE_REMOTE` capability then SWL Manager copies component files to remote processor update folder, otherwise, if an Update Agent has `E_SWL_UA_CAPS_BMASK_BACKUP_UPDATE_FILES` capability then SWL Manager copies component files to local backup folder that will be used on the post-install phase. If an Update Agent has `E_SWL_UA_CAPS_BMASK_UPDATE_REMOTE`, it means that the Update Agent itself executes on a remote processor and SWL Manager will transfer images to the remote processor for updates. When both capabilities are present, `E_SWL_UA_CAPS_BMASK_BACKUP_UPDATE_FILES` has priority over `E_SWL_UA_CAPS_BMASK_UPDATE_REMOTE`.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.3 GetTimeout

Type: DBus Method

*Implementation **Optional.***

Parameter	Type	Direction	Description
`timeout_state`	UINT32 "i"	input	The state of the timeout required as defined by `tpSWL_UaTimeoutStates_t`
`timeout`	UINT64 "t"	output	Time in seconds of how long SWL Manager should wait for the update agent.

This DBUS method reports the time that Software Loading Manager will wait for an Update Agent Status response. The possible values to use for the timeout_state are defined in `tpSWL_UaTimeoutStates_t`. If the Update Agent doesn't have specific needs to specify a particular timeout, the Update Agent should return an error so that Software Loading manager will use the default timeout.

If an Update Agent has `E_SWL_UA_CAPS_BMASK_GET_TIMEOUT` capability, the Update Agent will provide the timeout value to Software Loading Manager. Otherwise, Software Loading Manager will get the default timeout value from persistency.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method and Software Loading Manager should use the default value for the timeout. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.2.4 CheckPrerequisites

Type: DBus Method

*Implementation **Required**.*

Parameter	Type	Direction	Description
`component_name`	STRING "s"	Input	Unique identifier referencing to the content type that the Update Agent can handle. `component_name` is analogous to **content type** concept and is included in the `content_type_support` parameter of the `GetRegistrationInformation` DBus Method.
`component_location`	STRING "s"	Input	Absolute path where the image file (associated with the **content type**) is located in the filesystem.
`component_attributes_location`	STRING "s"	Input	**update_package_descriptor.xml** file containing metadata about the image file and its associated **content type**.

This DBus method is the entry point of the Update Agent to verify the integrity of a component in regards to the update process. An Update Agent takes `component_name` and `component_location` parameters to identify the object that needs to be verified. Using the `component_attributes_location` parameter, which is the **update_package_descriptor.xml** file, it gathers extra information that can help for the component verification. If the verification fails, it means that the prerequisites are not satisfied, therefore the update process fails at this point. The Update Agent should ensure that prerequisites check process should be run without affecting the currently running software and ensure that the component being updated is going to be functional once the update is applied.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.2.5 StartUaUpdate

Type: DBus Method

*Implementation **Required**.*

Parameter	Type	Direction	Description
`component_name`	STRING "s"	Input	Unique identifier referencing to the content type that the Update Agent can handle. `component_name` is analogous to **content type** concept and is included in the `content_type_support` parameter of the `GetRegistrationInformation` DBus Method.
`component_location`	STRING "s"	Input	Absolute path where the image file (associated with the **content type**) is located in the filesystem.
`component_attributes_location`	STRING "s"	Input	**update_package_descriptor.xml** file containing metadata about the image file and its associated **content type**.
`update_type`	INT32 "i"	Input	Update type: seamless software update (SSU) defined by the enumeration `tpSWL_UpgradeMode_t`.

This DBus method performs the following operation depending on `update_type` parameter value:

- E_SWL_UPGRADE_MODE_SSU: Prepare the images to be updated. The update is not done in place (modifying directly the current software), instead, the Update Agent performs setup operation to make sure that the update is ready for whenever is requested. The operations that an Update Agent could perform may involve creating backup copies of current software, verifying that the new images are valid, executing the update process in temporal image copies, among other operations. It is responsibility of the Update Agent to finish all the preparations at this point before next call is invoked. Note that by default it is expected that Update Agents support this mode.

It takes `component_name` and `component_location` parameters to identify which **content type** and image file should be used for the update process. `component_attributes_location` parameter provides further metadata to help with the update process, finally the update_type is specified in case the Update Agent requires to perform different tasks according to the upgrade mode defined by the enumeration `tpSWL_UpgradeMode_t`.

If the component supports Double Banking, the inactive bank of that component is updated. If the component only supports single banking, a backup of the current component is created before the update.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.2.6 GetAttributeRequest

Type: DBus Method

*Implementation **Optional**.*

Parameter	Type	Direction	Description
`installer_type`	UINT32 "u"	Input	SCOMO installer type.
`component_id`	STRING "s"	Input	SCOMO installer unique identifier.
`attribute_type`	UINT32 "u"	Input	SCOMO attribute type to be queried as defined in `tpSWL_ComponentAttribute_t`
`attribute_data`	STRING "s"	Output	Based on the `attribute_type` query, it return a string with the information required.
`is_comp_missing`	UINT32 "u"	Output	Unused at this moment.

This DBus method populates information about the Update Agent. It takes `installer_type, component_id` and `attribute_type` parameter to perform a query to the Update Agent and return the requested attribute information. In order to have a successful interaction, the Update Agent should understand what kind of requests can received from the calling function and adapt a proper answer to each request.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.7 PrepareForReset

Type: DBus Method

*Implementation **Required**.*

Parameter	Type	Direction	Description
`Not Applicable`	Not Applicable	Not Applicable	Not Applicable

This DBus method notifies the Update Agent that a software reset is about to happen and it should be prepared for that. Depending on the Update Agent, the arrangements might be changing an operation mode, clearing persistent flags, etc. This method is called by SWL Manager.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.2.8 UaVerifyImages

Type: DBus Method

*Implementation **Optional**.*

Parameter	Type	Direction	Description
`image_name`	STRING "s"	Input	Unique identifier referencing to the image that the Update Agent supports.

This DBus method requests Update Agent to retrieve the CRC and the version for the specified component.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.9 UaUpdateOutcome

Type: DBus Method

*Implementation **Required**.*

Parameter	Type	Direction	Description
`Result`	UINT32 "u"	Input	Passes the result of an update to the Update Agent as stated on the enumeration `tpSWL_VerificationResult_t`.
`reset_needed`	BOOLEAN "b"	Output	Tells the Software Loading Manager that the Update Agent requires a reset after having performed tasks given certain update outcome.

This DBus method allows the Update Agent to perform specific tasks depending on an update outcome and allows to request a reset to the Software Loading Manager. The update outcome is passed to the Update Agent through the `result` input argument and the reset request is done through the `reset_needed` output argument.

The Update Agent should perform different operations based on the value of `result` parameter as defined by `tpSWL_VerificationResult_t`

- E_SWL_RESULT_ACCEPT: The update has been accepted. The new images can be used. Perform post-install operations to finalize the update as needed. Update Agents can return failure if such post-install operations fail at this point, this would fail the overall update process. **Due to this, Update Agents are requested to keep all the needed files to perform rollback operations to the original image at this point, even if they have received the update acceptance notification and returned success. This is so that a rollback can be performed if other update agents report update failed for the same campaign. To consider the update campaign a success and use the updated images, all update agents have to report success for all elements being updated.**
- E_SWL_RESULT_REJECT: The update has been rejected. The new images should be rollbacked and old images should be restored. Update Agents can return failure if such operations fail at this point, this would leave the whole system in an irrecoverable state.
- E_SWL_RESULT_TIMEOUT: Update approval or rejection was not received on time. Perform same operations as with E_SWL_RESULT_REJECT.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

Figure 4 One Update Agent fails in the Update Outcome phase shows that in case of failure, all update agents have to rollback and restore the old image. To accomplish this, the old image needs to be available during the Update Outcome phase (i.e. Update agent shall not delete/overwrite it). Note that clean up tasks (i.e. restoration of the updatable elements into the original state) shall be performed after the rollback and during the Complete Update phase.

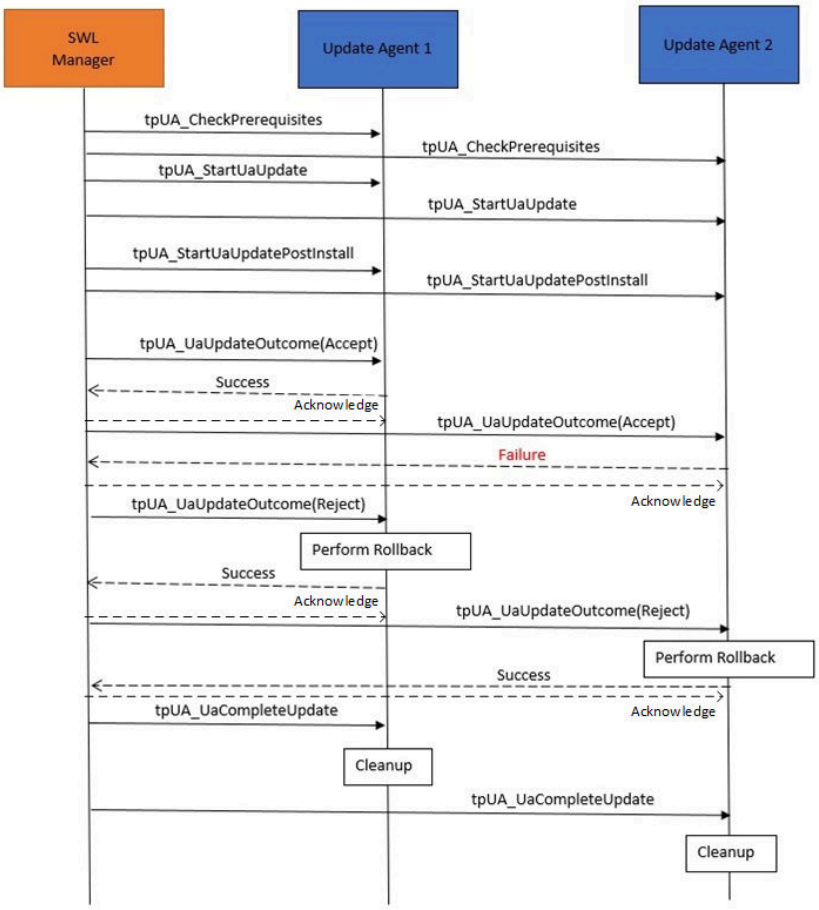


Figure 4 One Update Agent fails in the Update Outcome phase

3.1.2.10 UaCompleteUpdate

Type: DBus Method

*Implementation **Required**.*

Parameter	Type	Direction	Description

© Continental Automotive GmbH 2019
Confidential and Proprietary

Not Applicable	Not Applicable	Not Applicable	Not Applicable

This DBus method notifies the Update Agent that a software update has been completed; at this point the Update Agent is required to perform its respective cleanup tasks. All persistent data used for current update operation should be cleared so a new update process can be started cleanly.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.11 UaCompleteUpdateWithVerification

Type: DBus Method

*Implementation **Required**.*

Parameter	Type	Direction	Description
Not Applicable	Not Applicable	Not Applicable	Not Applicable

This DBus method notifies the Update Agent, with double banking support, that a software update has been completed with an integrity verification check, which consists on comparing the CRC's of the primary and secondary banks after the bank synchronization. The Update Agent is required to perform its respective cleanup tasks. All persistent data used for current update operation should be cleared so a new update process can be started cleanly.

This interface can be used only if `E_SWL_UA_CAPS_BMASK_VERIFY_COMPLETE_UPDATE` and persistence key `swl_complete_update_verification_check_enable` are enabled (refer to TP_F101U-00028).

Return values: If the CRC's of the compared banks are equal, `E_SWL_ERROR_NONE` is returned. In case of failure, or if the CRC's of the compared banks are different, a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.12 UaPrepareForUpdate

Type: DBus Method

*Implementation **Required**.*

Parameter	Type	Direction	Description
`component_name`	STRING "s"	Input	Unique identifier referencing to the content type that the Update Agent can handle. `component_name` is analogous to **content type** concept and is included in the `content_type_support` parameter of the `GetRegistrationInformation` DBus Method.

© Continental Automotive GmbH 2019
Confidential and Proprietary

component_location	STRING "s"	Input	Absolute path where the image file (associated with the **content type**) is located in the filesystem.
component_attributes_location	STRING "s"	Input	**update_package_descriptor.xml** file containing metadata about the image file and its associated **content type**.
update_type	INT32 "i"	Input	Update type: seamless software update (SSU) defined by the enumeration tpSWL_UpgradeMode_t.

This DBus method notifies the Update Agent that a software update will be triggered by a following StartUaUpdate. The Update Agent can perform tasks prior to the update starting. The arguments for this method are same as the ones for StartUaUpdate with the same purpose.

Return values: On success, E_SWL_ERROR_NONE is returned. In case of failure a value indicative of the failure of type tpSWL_Error_t should be used in the DBus error return method. If there is no error code defined in tpSWL_Error_t type to properly identify the error, then a value of E_SWL_ERROR_GENERIC should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.13 StartUaUpdatePostInstall

Type: DBus Method

*Implementation **Required.***

Parameter	Type	Direction	Description
component_name	STRING "s"	Input	Unique identifier referencing to the content type that the Update Agent can handle. component_name is analogous to **content type** concept and is included in the content_type_support parameter of the GetRegistrationInformation DBus Method.
component_location	STRING "s"	Input	Absolute path where the image file (associated with the **content type**) is located in the filesystem.
component_attributes_location	STRING "s"	Input	**update_package_descriptor.xml** file containing metadata about the image file and its associated **content type**.
update_type	INT32 "i"	Input	Update type: seamless software update (SSU) defined by the enumeration tpSWL_UpgradeMode_t.

© Continental Automotive GmbH 2019
Confidential and Proprietary

			Expected to be E_SWL_UPGRADE_MODE_SSU.

This DBus method performs the update process for SSU workflow. This API will be called after StartUaUpdate one. An Update Agent is expected to finalize the update process that was prepare during StartUaUpdate API invocation. It takes `component_name` and `component_location` parameters to identify which **content type** and image file should be used for the update process. `component_attributes_location` parameter provides further metadata to help with the update process, finally the update_type is specified in case the Update Agent requires to perform different tasks according to the upgrade mode.by the enumeration `tpSWL_UpgradeMode_t`, nevertheless note that this DBus method is only intended for E_SWL_PHASE_POST_INSTALL update type, therefore it is expected that Update Agent uses this value.

Note that this method will be invoked for all update agents. Update agents that support double banking may immediately return a success indication with no further processing. Update agents that support single banking must process the request and finalize the update process as appropriate.Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.14 UaVerifySoftwareActivation

Type: DBus Method

Implementation Required.

Parameter	Type	Direction	Description
`content_type`	STRING "s"	Input	The type of content updatable by this Update Agent.
`verification_status`	INT32 "i"	Output	Software activation verification status as defined on the enumeration `tpSWL_SoftwareActivationResult_t`.

This DBus method performs the Software Activation Verification for the Update Agent registered with the specified `content_type` in current update operation.

The result should be a any of the values as defined on the enumeration `tpSWL_SoftwareActivationResult_t`.

An Update Agent should consider the following points for the proper implementation of this DBus method:

- Whenever an Update Agent handles multiple `content_type` values, it should be capable of handling multiple `UaVerifySoftwareActivation` calls for each `content_type` involved in the update process.
- The operation is synchronous and the Update Agent should finish its verification before the defined DBus timeout value expires.
- The Update Agent may be called more the one time among different boot cycles.
    - The result should be focused in the current boot cycle and not recollected from previous boot cycles.
    - This is required with the purpose of confirming that the software has been activated after retrying with another reboot operation.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.15 UaGetImageData

Type: DBus Method

*Implementation **Optional.***

Parameter	Type	Direction	Description
`image_data`	ARRAY OF COMPLEX "a(us) "	Output	Array of image names and types supported by the Update Agent.This is defined by the structure `tpSWL_ImageData_t`. Image_type is defined by the enumeration `tpSWL_ImageType_t`

This DBus method requests Update Agent to retrieve a list of the supported images (components) and the image type of each of them.

Return values: On success, `E_SWL_ERROR_NONE` is returned. In case of failure a value indicative of the failure of type `tpSWL_Error_t` should be used in the DBus error return method. If there is no error code defined in `tpSWL_Error_t` type to properly identify the error, then a value of `E_SWL_ERROR_GENERIC` should be used to identify an internal logical error within the Update Agent implementation.

3.1.2.16 PrerequisitesStatus

Type: DBus [Deprecated] or Socket Signal

*Implementation **Required**.*

Parameter	Type	Description
`component_name`	STRING "s"	Component unique name.
`component_status`	INT32 "i"	Component update result.

This DBus [Deprecated] or Socket signal is a notification from the Update Agent to any listening process, normally the Software Loading Manager that the prerequisites process has finished.

Implementations are expected to set the values in the signal as follows.

`component_name` is the **content type** that identifies the element that was verified.

`component_status` is the status of the verification defined as an enumerated value from `tpSWL_UpdateStatus_t`

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.2.17 UaUpdateStatus

Type: DBus [Deprecated] or Socket Signal

*Implementation **Required**.*

Parameter	Type	Description
`component_name`	STRING "s"	Component unique name.
`component_status`	INT32 "i"	Component update result.

This DBus [Deprecated] or Socket signal is a notification from the Update Agent to the Software Loading Manager that the current update operation has finished. It is expected to be sent as a result of the operations triggered by StartUaUpdate and StartUaUpdatePostInstall DBus methods.

Implementations are expected to set the values in the signal as follows.

`component_name` is the identifier of the element that was updated. It should match with the component_name provided by StartUaUpdate and StartUaUpdatePostInstall DBus methods.

`component_status` is the status of the update process defined as an enumerated value from `tpSWL_UpdateStatus_t`.

3.1.2.18 UaPrepareForResetStatus

Type: Socket Signal

*Implementation **Required**.*

Parameter	Type	Description
`ua_name`	STRING "s"	*Update Agent* unique name.
`component_status`	INT32 "i"	Component update result.

This Socket signal is a notification from the Update Agent to the Software Loading Manager that the current operation before reset has finished. It is expected to be sent as a result of the operations triggered by PrepareForReset DBus methods.

Implementations are expected to set the values in the signal as follows.

`ua_name` is the identifier of the element that was updated. It should match with the component_name provided by StartUaUpdate and StartUaUpdatePostInstall DBus methods.

`component_status` is the status of the update process defined as an enumerated value from `tpSWL_UpdateStatus_t`.

3.1.2.19 UaUpdateOutcomeStatus

Type: DBus [Deprecated] or Socket Signal

*Implementation **Required**.*

© Continental Automotive GmbH 2019
Confidential and Proprietary

Parameter	Type	Description
`ua_name`	STRING "s"	Update Agent unique name.
`status`	INT32 "i"	Update Agent validation result.

This DBus [Deprecated] or Socket signal is a notification from the Update Agent to the Software Loading Manager that the current validation process has finished. It is expected to be sent as a result of the operations triggered by StartUaUpdateOutcome DBus method.

If `E_SWL_UA_CAPS_BMASK_UPD_OUTCOME_ASYNC` capability is enabled, Update Outcome will work asynchronously.

Implementations are expected to set the values in the signal as follows.

`ua_name` is the identifier of the element that was updated. It should match with the component_name provided by StartUaUpdate and StartUaUpdatePostInstall DBus methods.

`status` is the status of the validation proces defined as an enumerated value from `tpSWL_UpdateStatus_t`

Note:

If no Acknowledge is received from the Software Loading Manager after a Status Signal is sent, Update Agents needs to perform a retry until an Acknowledge is sent from the Software Loading Manager side.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.2.20 UaCompleteUpdateStatus

Type: DBus [Deprecated] or Socket Signal

*Implementation **Required**.*

Parameter	Type	Description
`ua_name`	STRING "s"	Update Agent unique name.
`status`	INT32 "i"	Component update result.

This DBus [Deprecated] or Socket signal is a notification from the Update Agent to the Software Loading Manager that the Complete Update operation process has finished. It is expected to be sent as a result of operation triggered by UaCompleteUpdate DBus method.

If `E_SWL_UA_CAPS_BMASK_COMPLETE_UPD_ASYNC` capability is enabled, Complete Update will work asynchronously.

Implementations are expected to set the values in the signal as follows.

`ua_name` is the identifier of the element that was updated. It should match with the component_name provided by StartUaUpdate and StartUaUpdatePostInstall DBus methods.

`status` is the status of the validation proces defined as an enumerated value from `tpSWL_UpdateStatus_t`

3.1.2.21 UaVerifyImagesResult

Type: DBus [Deprecated] or Socket Signal

*Implementation **Optional**.*

Parameter	Type	Description
`ua_name`	STRING "s"	Update Agent unique name.
`image_name`	STRING "s"	Image name.
`image_version`	STRING "s"	Version of the requested image.
`image_checksum`	UINT32 "u"	Checksum of the requested image.

This DBus [Deprecated] or Socket signal is a notification from the Update Agent to the Software Loading Manager that the retrieval of the CRC and the version for the specified image (component) has finished. It is expected to be sent as a result of the operations triggered by UaVerifyImages DBus method.

Implementations are expected to set the values in the signal as follows.

`ua_name` is the identifier of the Update Agent.

`image_name` is the identifier of the element supported by the Update Agent

`image_version` is the retrieved version of the provided image

`image_checksum` is the retrieved CRC of the provided image

© Continental Automotive GmbH 2019
Confidential and Proprietary

4 FAQs

Not applicable.

5 References

5.1 Abbreviations/Acronyms/Definitions

UA	Update Agent

5.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F101U-00028: User Guide Software Loading Manager

5.3 Website links

https://www.freedesktop.org/wiki/Software/dbus/
https://www.freedesktop.org/wiki/Software/systemd/

© Continental Automotive GmbH 2019
Confidential and Proprietary

6 Revision History

Version	Date	Description
01.00	Aug 15, 2019	Initial release
02.00	Dec 4, 2019	Remove all of "recovery mode" references Changed classifications for GetCapabilities, GetAttributeRequest, StartUaUpdatePostInstall.
03.00	Mar 13, 2020	Update and add sequence diagram in UaUpdateOutcome API description
04.00	Mar 25, 2020	Update naming convention for Update Agent DBUS Service
05.00	Jun 19, 2020	Added Double Banking support changes.
06.00	Jul 6, 2020	Software Activation Verification
07.00	Jul 1, 2021	Socket interfaces and retry mechanism
08.00	Jul 9, 2021	Usage of swl_allow_same_version_write persistence key
09.00	Sep 30, 2021	Add interface UaCompleteUpdateWithVerification. Add GetTimeout DBUS interface and details.
		== ConMod-specific content from here ==

© Continental Automotive GmbH 2019
Confidential and Proprietary